UNIFIED ENGINEERING WORKBENCH
TECHNICAL REPORT

Mansi Nahar S, mn4866@srmist.edu.in

1. Project Overview

The Unified Engineering Workbench is a modular C++17 application accompanied by a lightweight browser-based dashboard. It brings together numerical heat-transfer analysis, geometry inspection, performance benchmarking, embedded-interface demonstrations, and a small neural-network backpropagation example. The project is intended as an engineering prototype that demonstrates how these capabilities can be organized, executed, and verified within one workbench.

The delivered application includes command-line modules and a static frontend. The dashboard presents the available modules and their example results; it does not currently communicate with a live backend or execute simulations directly from the browser.

2. Software Architecture

The core is implemented in C++17 and organized into separate modules so that each engineering function can be built and tested independently. CMake is used to configure and build the project. A Windows launcher opens the dashboard from the packaged frontend directory, while the command-line executable runs individual modules.

- Thermal module: performs a one-dimensional heat-transfer calculation and reports midpoint error for increasing mesh resolutions.
- Geometry module: reads the supplied STL sample and reports triangle count and axis-aligned bounds.
- Performance module: compares execution time across worker counts and reports speedup, efficiency, and output difference.
- Hardware module: demonstrates device discovery and telemetry using simulated hardware interfaces.
- Backpropagation module: computes a small network's gradients and compares them with finite-difference estimates.
- Frontend: provides a static dashboard with sections for the workbench modules and their results.

3. Thermal Finite Element Method Demonstration

The thermal demonstration solves a one-dimensional steady heat-conduction problem using a discretized numerical model. The program evaluates the numerical solution at the midpoint and compares it with the analytical reference, reporting the maximum midpoint error for each mesh size.

The results show that the error decreases as the number of elements increases. When the mesh is doubled, the reported error decreases by approximately a factor of four, which is consistent with the expected convergence behaviour of a second-order spatial approximation for this test case.

Mesh size (N)	Maximum midpoint error (°C)
10	1.25
20	0.3125
40	0.078125
80	0.01953125

4. Geometry and Data Validation

The geometry module was executed using the supplied sample STL file. It successfully identified four triangles and calculated the minimum and maximum coordinate values along each of the three axes. The supplied temperature CSV file was also processed, and the validator reported 11 validated nodes.

Geometry output	Observed value
Triangle count	4
Axis 0 minimum / maximum	0 / 1
Axis 1 minimum / maximum	0 / 1
Axis 2 minimum / maximum	0 / 1
Validated CSV nodes	11

5. Multicore Performance Benchmark

The performance module was run with one, two, and four workers. The table below records the latest packaged Windows run. The two-worker configuration was fastest in this run. Four workers also improved on the single-worker result, but did not outperform two workers. The maximum output difference was zero degrees Celsius for all tested configurations.

Workers	Median time (ms)	Speedup	Efficiency	Max difference (°C)
1	2.6127	1.00000	1.00000	0
2	2.0496	1.27474	0.637368	0
4	2.1095	1.23854	0.309635	0

Execution times are dependent on the machine, operating-system scheduling, and background workload. These measurements should therefore be treated as results from this particular run rather than universal performance guarantees.

6. Embedded Interface Demonstration

The hardware module demonstrates the structure of an embedded-device interface using software mocks. It lists representative devices and their interface characteristics, then produces simulated telemetry from an STM32 Nucleo mock. A timeout scenario is also demonstrated to show how the application reports a recoverable communication failure.

No physical microcontroller, FPGA board, or USB-JTAG adapter was connected during this verification. All device names, telemetry, and communication behaviour in this module are simulated.

Mock device	Category	Demonstrated interface
STM32 Nucleo (mock)	Microcontroller	USB CDC telemetry; 3.3 V GPIO
Basys 3 (mock)	FPGA	JTAG configuration/debug; 3.3 V I/O
USB-JTAG adapter (mock)	Debug adapter	TCK/TMS/TDI/TDO; host USB

7. Backpropagation and Numerical Verification

The backpropagation demonstration evaluates a small neural-network calculation, computes gradients for its parameters, applies a sample update, and compares the analytical gradients with finite-difference estimates. The close agreement between the analytical and numerical values provides a check on the gradient implementation.

Parameter	Analytical gradient	Finite-difference gradient
w	-1.20946447524	-1.20946447513
b	-1.20946447524	-1.20946447513
v	-0.710682047486	-0.710682047433
c	-1.53788284274	-1.53788284263

The demonstration reported h = 0.46211715726, yhat = 0.46211715726, and loss = 1.182541819. The parameter update used a learning rate of 0.1, producing updated values w = 0.620946447524, b = 0.120946447524, v = 1.07106820475, and c = 0.153788284274.

8. Build, Test and Package Verification

The project was configured and built on Windows using CMake and the Visual Studio C++ toolchain. The Release configuration compiled successfully. CTest completed with one test passed and zero tests failed.

The Windows packaging script recreated the distribution directory and copied the launcher, command-line executable, frontend, sample data, documentation, and result files. The packaged launcher was executed and successfully opened the dashboard. The packaged command-line executable was also run successfully.

The following commands were used from a Developer PowerShell for Visual Studio terminal:

```
cmake -S . -B build -A x64
cmake --build build --config Release
ctest --test-dir build -C Release --output-on-failure
.\dist\bin\uew.exe thermal
.\dist\bin\uew.exe geometry
.\dist\bin\uew.exe backprop
.\dist\bin\uew.exe hardware
```

9. Limitations and Future Work

- The dashboard is currently a static frontend and is not connected to a live simulation backend.
- The geometry demonstration reports basic mesh statistics; it does not provide interactive three-dimensional visualization or advanced mesh-quality analysis.
- The embedded-device interfaces are software mocks. Physical-device integration and hardware-in-the-loop testing remain future work.
- The sample temperature data is provided for demonstration and validation, not as measured experimental data.
- The benchmark results are specific to the test environment and should be repeated on target hardware for meaningful performance comparisons.

Future development could connect the dashboard to the C++ modules through a local service, add interactive geometry rendering, integrate physical embedded devices, and expand the numerical validation suite.

10. Dependencies and Reproducibility

The core application uses C++17 and CMake and was built with the Visual Studio C++ toolchain on Windows. The project includes its sample input data and build script. The dashboard uses local HTML, CSS, and JavaScript files. The packaged application can be launched from the dist directory after a successful build.

Before final submission, the dependency list and third-party code declarations should be checked against the actual repository contents and any libraries or tools used during development. This report does not claim that no third-party components were used.

11. Conclusion

The Unified Engineering Workbench brings several engineering demonstrations into one modular application and provides a packaged Windows interface for viewing and running them. The thermal results show decreasing numerical error with mesh refinement; geometry and CSV checks complete successfully; the benchmark demonstrates measurable parallel performance; the backpropagation gradients agree with finite-difference estimates; and the hardware mock demonstrates telemetry and timeout handling. The build, automated test, and packaged launcher were all verified in the Windows environment.

The workbench is a prototype rather than a production engineering platform. Its current limitations, particularly the static dashboard and simulated hardware interfaces, are documented so that the scope of the demonstrated functionality is clear.

Submission Note

The accompanying ownership statement and AI-use disclosure should be completed and reviewed by the candidate. Any description of individual contributions, third-party code, dependencies, and verification activities must reflect the work actually performed.